

Predicting the winner of a Counter-Strike 2 round

Exploratory data analysis, PCA and logistic regression on a dataset of professional matches

Lucas Lacharme • May 27, 2026

Abstract

Counter-Strike 2 is a 5-on-5 tactical shooter in which a team's economy and overall skill level largely determine the outcome of each round. From a dataset of **1,511 professional matches** and **~32,000 rounds** (Tier 1 tournaments, 2025–2026) built by combining HLTV scraping with parsing of official demos, we study how predictable the round winner is from the game state observed just before the round begins (equipment, money, score, momentum). Three regimes (**pistol**, **post-pistol**, **normal**) are analysed separately through correlation, PCA and logistic regression.

1 Motivation

Counter-Strike 2 (*CS2*) pits a team of *Counter-Terrorists* (CT) against a team of *Terrorists* (T) over 24 rounds (MR12 format, first team to 13 round wins). The T side tries to plant a bomb on one of two sites; the CT side either prevents the plant or defuses the bomb once it is down. Sides are swapped at half-time. Before each round, a *freeze-time* phase lets players buy weapons, armour and grenades under an economic system whose payouts depend on previous rounds. This rule set produces an asymmetry from one round to the next and makes the *game state measured right before the round begins* potentially very informative about its outcome.

The central question is therefore:

“Knowing the game state (equipment, money, score, momentum, rating and recent form) at the instant a round starts, can we predict which team is going to win that round?”

2 Data collection

No public source provides, for CS2, a round-level dataset aligned with HLTV team statistics. We therefore build it by combining two streams: the *demos* (official replays downloaded from HLTV) from which we extract, **round by round**, information about the game state at the start of the round (equipment, money, weapon and utility counts, etc.), and the *team statistics* scraped from HLTV.org (rating, ADR, round-win %, flash-assists, etc., broken down by map, side, time window 30 d/90 d/6 months and opponent tier). Figure 1 summarises the pipeline; the technical details (snapshot tick, cleaning steps, NaN handling) are reported in appendices A to C.

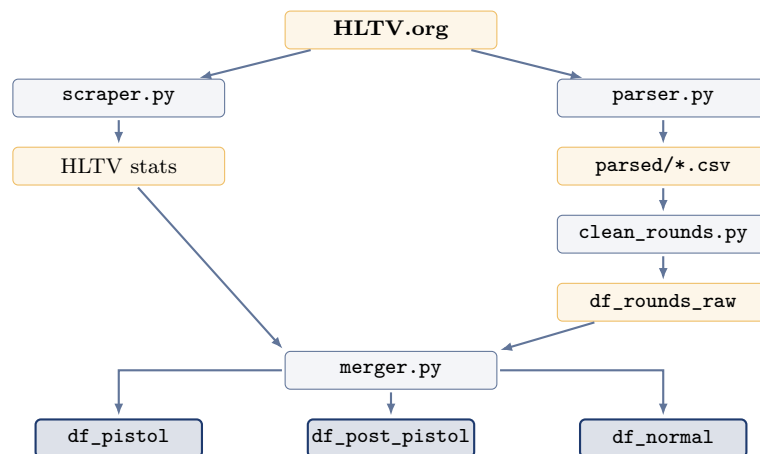


Figure 1: Data collection pipeline.

Variables produced. Table 1 groups the variables available after the join by family: *demo* variables are duplicated per side (**ct_***, **t_***) and *HLTV* statistics provide 18 to 24 indicators per team depending on the split. The target is **round_winner** (1 if CT wins, 0 otherwise).

Family	Main variables
Economy	money_total, cash, equipment_value, utility_value, loss_count
Weapons	awp_count, rifle_count, smg_count, heavy_count, ak_count (T)
Armour	armor_count, helmet_count, defuser_count (CT)
Utility	smoke_count, flash_count, he_count, molo_count
Context	round_number, score, survivors_previous, previous_bomb_planted
Momentum	rounds_won_streak, rounds_lost_streak
HLTV stats	rating, ADR, round-win %, flash-assists, etc., broken down by map, side, time window and opponent tier
Identifiers	match_file, map_name, match_date, round_winner (target)

Table 1: Variable families after the join.

3 Three round regimes

CS2 rounds are not equivalent. **Pistol rounds** (R1, R13) start from an economy reset to \$800 per player: money and weapon variables are nearly constant. **Post-pistol rounds** (R2, R14) display a structural economic asymmetry caused by the pistol outcome (winner on a *full-buy*, loser on *eco/force-buy*). **Normal rounds** cover the rest of the game, where each team freely chooses its buy depending on accumulated cash, and where all buy types (eco, half-buy, force-buy, full-buy) coexist. Training a single model on these three regimes would mix three very different distributions; we therefore separate the dataset into three splits at join time (Table 2). All subsequent analysis (correlations, PCA, regression) is carried out separately on each split.

Split	Rounds	Columns	CT win rate
df_pistol	2,945	70	50.5 %
df_post_pistol	2,956	105	52.3 %
df_normal	26,322	106	51.1 %

Table 2: Final partition of rounds.

3.1 Pearson correlations with round_winner

Our target is `round_winner`, coded $Y = 1$ if CT wins the round and $Y = 0$ otherwise. To quickly assess the strength of the linear link between each feature X and Y , we use the Pearson correlation coefficient, bounded in $[-1, +1]$:

$$r_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}.$$

For each round we form the *deltas* $\Delta X = X^{\text{CT}} - X^{\text{T}}$ of the economic and HLTV features, and we compute r between each ΔX and Y on each of the three splits (Figure 2).

Profiles are very contrasted. In **pistol** rounds all correlations stay below $|r| < 0.06$: no individual feature discriminates strongly. In **post-pistol** rounds the economic deltas (`equipment_value`, `money_total`, `utility_value`, weapon counts) reach $|r| \approx 0.6$, a direct signature of the asymmetry created by the pistol outcome. In **normal** rounds the same deltas remain at the top but with a more moderate amplitude ($|r| \approx 0.3$). Across all three regimes we observe a clear separation between the two feature families: parser-derived economic variables carry most of the signal while HLTV statistics (rating, ADR, round-win %, ...) remain systematically at a lower correlation level.

4 Analysis per regime

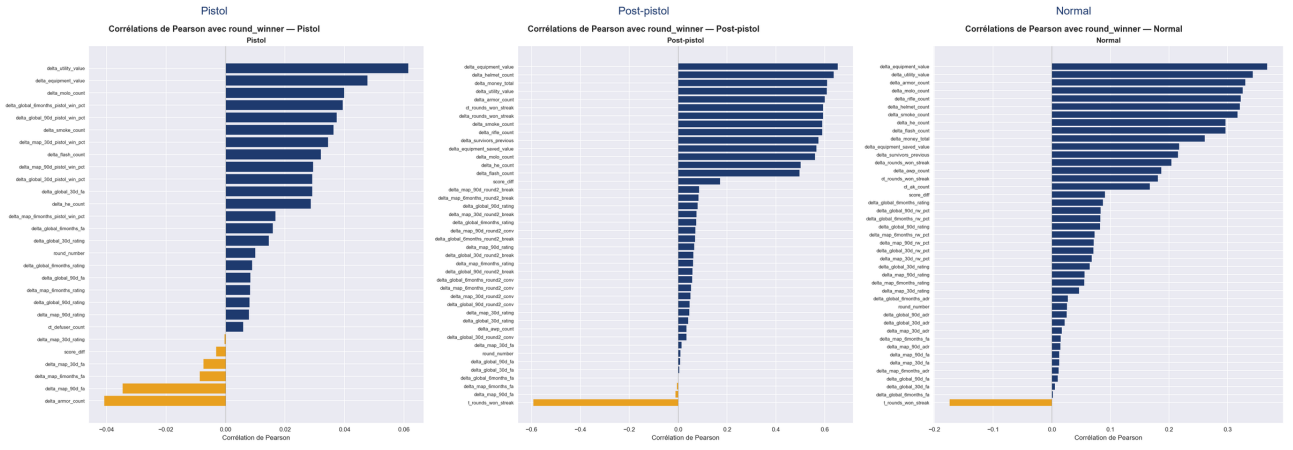


Figure 2: Pearson correlations between each CT – T delta and `round_winner`, by regime. Blue: positive correlation (feature pushes toward a CT win); orange: negative.

4.1 Pistol

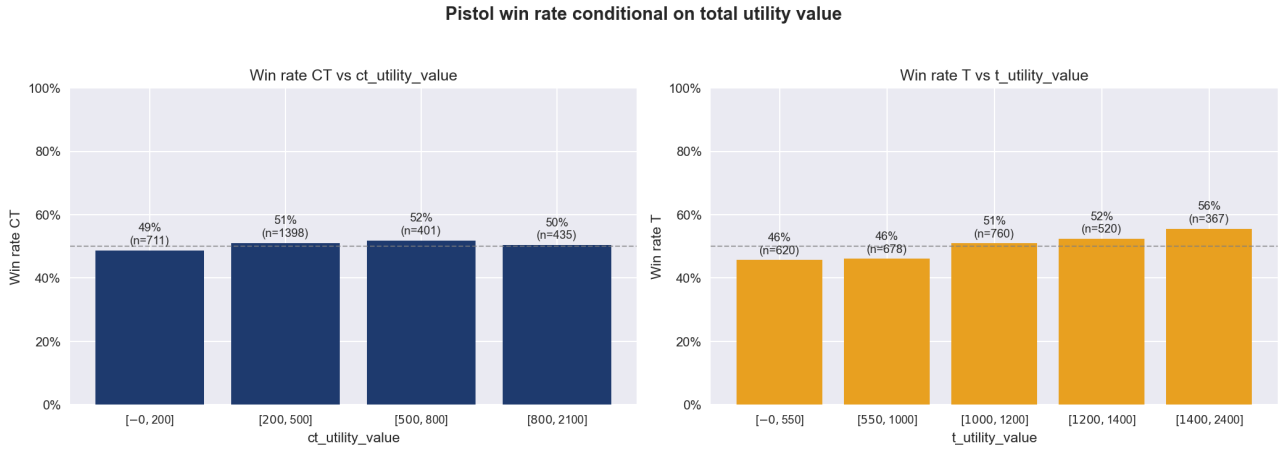


Figure 3: Pistol win rate conditional on the total `utility_value` purchased by the team, per quintile.

In pistol rounds, budgets are nearly equal and tightly bounded (\$800/player), so very few features discriminate: all correlations stay below $|r| < 0.06$. The only individual lever worth exploiting is the **armour** ↔ **utility** trade-off on the T side, where skipping kevlar (\$650) to fund utility raises the win rate by about 10 points (Figure 3). The **map** contributes a signal of comparable amplitude (~8 points between `de_dust2` and `de_anubis`). HLTV statistics remain uninformative: the historical form of a team does not transfer to pistol rounds, which behave almost like a coin flip.

4.2 Post-pistol

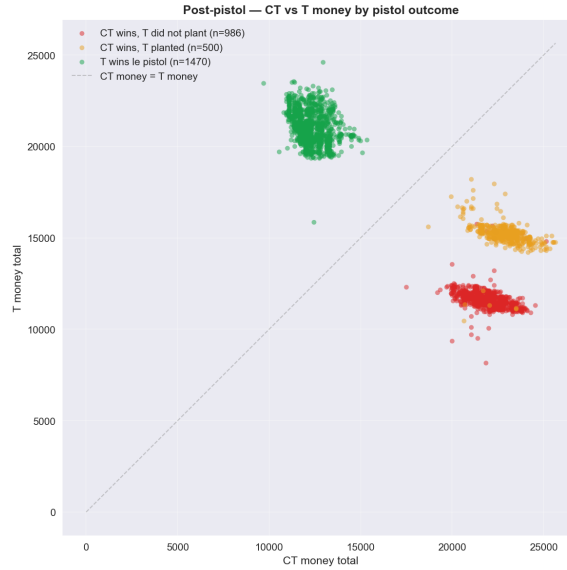


Figure 4: Post-pistol: `CT money_total` vs `T money_total`, coloured by pistol outcome. The asymmetry is sharp — pistol winner $\approx$ full buy, pistol loser $\approx$ eco/force.

The pistol outcome creates a structural **economic asymmetry** between the winning team, which full-buys, and the losing team, whose limited cash only allows an eco or a force-buy (Figure 4). This asymmetry translates into **contrasted buy decisions** on the losing side: a near-systematic force-buy on the CT side after a lost pistol (78 %), versus only 48 % on the T side. The T decision is actually conditioned by the *plant* of the bomb in the previous round. Its effect is directly measurable on the force-buy profitability: without a plant the T force-buy wins 26 % of the time; with a plant the +\$600 per player bonus is enough to lift it to 41 %. This chain *asymmetry* $\rightarrow$ *decision* $\rightarrow$ *outcome* gives post-pistol its exploitable signal, strongly correlated with the pistol outcome.

4.3 Normal

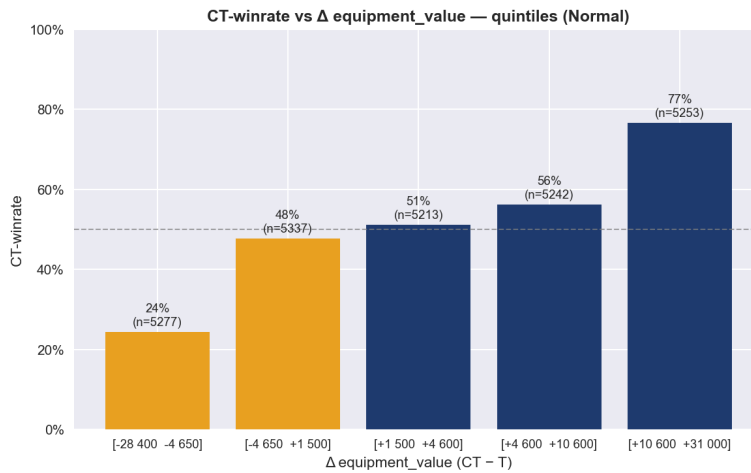


Figure 5: CT win rate on normal rounds conditional on $\Delta \text{equipment_value}$ (CT - T), per quintile.

On normal rounds, all buy types coexist (eco, half-buy, force-buy, full-buy) and the dominant signal is the economic gap measured directly by $\Delta \text{equipment_value}$: its lowest quintile corresponds to a win rate around 24 % and its top quintile to 77 % (Figure 5). At comparable economy (mutual full-buy with `equipment_value` > 20 k on both sides), two secondary signals remain. The $\Delta \text{awp_count}$ adds about 5 points of win rate per extra AWP; the map-specific HLTV rating still separates teams but

with a moderate amplitude (win rate bounded in [47 %, 55 %] between extreme quintiles).

5 Principal Component Analysis

PCA is applied separately to the three splits, on the CT – T *deltas* of the economic and HLTV features, after standard scaling (`StandardScaler`). We pick `StandardScaler` over a robust variant because the upstream cleaning steps (corrupted demos, phantom rounds, 4v5 pistols) have already removed the structural outliers. The aim is to test whether an effective dimensionality reduction is possible before the regression.

Regime	Axes for 80 % inertia / initial features
pistol	12 / 70
post-pistol	12 / 105
normal	10 / 106

Table 3: PCA summary by regime.

None of the three regimes lends itself to a useful reduction: 10 to 12 axes are needed to capture 80 % of cumulative inertia, i.e. about 15 % of the initial features. We therefore keep the full feature set for the logistic regression.

6 Logistic regression

For each regime, we model the probability that CT win the round through a logistic regression:

$$P(Y = 1 \mid X = x) = \sigma(\alpha + \beta^\top x), \quad \sigma(t) = \frac{1}{1 + e^{-t}},$$

where $Y = 1$ indicates a CT win. The train/test split is **temporal and grouped by match**: matches are sorted by `match_date` and the most recent 20 % form the test set (80/20). This scheme preserves match integrity (no round from the same match is split between train and test) and reproduces the conditions of a real prediction setup (past → future). Features are standardised on the training set only. We compare a baseline (majority class) to four nested feature configurations: **A** (round-state only), **B** = A + HLTV, **C** = B + map (one-hot), and a case **D** that adds quasi-target variables (`rw_pct`, `pistol_win_pct`, round-2 indicators) in order to measure the effect of including variables that already partly encode the target. We report **accuracy** and **AUC** on the test set.

AUC (accuracy in parentheses)	Pistol	Normal	Post-pistol
Baseline (majority class)	0.50 (.50)	0.50 (.51)	0.50 (.53)
A — round-state	0.54 (.54)	0.70 (.63)	0.86 (.78)
B — A + HLTV	0.55 (.54)	0.71 (.64)	0.85 (.76)
C — A + HLTV + map	0.56 (.51)	0.71 (.64)	0.85 (.76)
D — + quasi-target features	<i>0.56</i> (.54)	<i>0.71</i> (.64)	<i>0.85</i> (.76)

Table 4: AUC and accuracy on the test set, by regime and feature configuration.

Three levels of predictability emerge:

$$\text{pistol AUC} \approx 0.56 < \text{normal AUC} \approx 0.71 < \text{post-pistol AUC} \approx 0.86.$$

The pistol regime stays barely above chance, in line with the absence of a strong individual signal observed in Section 3. On the other two regimes, the *round state* alone (case A) already captures most of the predictive power. The HLTV, map and even quasi-target additions **add little** (at most +0.01 AUC on the normal and post-pistol regimes): the carrying signal is concentrated in the immediate economic and material state, and the global team indicators only complement it marginally.

7 Conclusion and outlook

This study first shows that not all rounds are equally easy to predict. In pistol rounds, the tightly bounded economy on both sides makes the outcome largely random ($\text{AUC} \approx 0.56$). In post-pistol rounds, on the contrary, the asymmetry created by the previous round allows an almost deterministic prediction ($\text{AUC} \approx 0.86$). The normal regime sits in between ($\text{AUC} \approx 0.71$). Within each regime, the immediate state of the round — equipment, money, weapons, utility — carries most of the predictive signal. Global team statistics (HLTV rating, recent form) and the map effect add little in comparison: they act as secondary levers that improve models only marginally.

Outlook. Three natural directions extend this work:

- *Models*: tuning the logistic regression penalty (ℓ_1 , ℓ_2 , ElasticNet), non-linear models (XGBoost).
- *Features*: enriching the dataset with individual player statistics, matchups, per-player form.
- *Vision*: integrating *live* signals (player positions, utilities thrown, strategy currently in use) for real-time prediction.

A Counter-Strike 2: rules and economy

A.1 Match format

The competitive mode studied in this project is *Bomb Defusal Competitive*. A match pits two five-player teams in **MR12** format: the 24 regulation rounds are split into two halves of 12 rounds, and the first team to reach **13 round wins** takes the match. On a 12-12 tie a 6-round *overtime* is played.

Teams swap sides at half-time (R13) and every three rounds during overtime. Each round starts with a 15-second *freeze-time* during which players can buy equipment but cannot move. The action phase then runs for a maximum of **1 minute 55 seconds**. Buys remain allowed for about 20 extra seconds as long as the player stays in their buy zone, so the total buy window lasts about 35 seconds.

A.2 Round objectives

The **Counter-Terrorists** win the round by:

- letting the round timer run out without the bomb being planted;
- defusing the bomb after it has been planted (10 seconds without a *defuse kit*, 5 seconds with);
- eliminating all Terrorists *before* the bomb is planted.

The **Terrorists** win the round by:

- planting the bomb on either site and letting the fuse run out (**40-second** fuse);
- eliminating all Counter-Terrorists.

If the bomb is planted, the usual rule "eliminate the whole enemy team" no longer applies to CT: even after killing all five Terrorists they still have to defuse the bomb to win the round.

A.3 Economy system

The CS2 economy fully structures strategic decisions. Each player has a personal account that they top up round after round and spend at the start of each round. Money is never lost on death or defeat; it only decreases on a purchase. The cap is \$16,000 per player.

Equipment categories and prices (CS2 Competitive). Table 5 summarises the main equipment prices, as implemented in the parser.

Category	Weapon / item	Price	Notes
Defensive equipment	Kevlar	\$650	Armour only
	Kevlar + Helmet	\$1,000	Only the helmet blocks one-tap HS by a pistol
	Defuse Kit	\$400	CT only, defuse 10 s $\rightarrow$ 5 s
	Zeus x27	\$200	Taser, guaranteed kill at contact
Starting pistols	Glock-18 (T) / USP-S, P2000 (CT)	0	Given at the start of every round
Pistols	Dual Berettas, P250	\$300	
	Tec-9 (T), Five-SeveN (CT), CZ75	\$500	
	R8 Revolver	\$600	
	Desert Eagle	\$700	One-tap HS at any range
SMGs	MAC-10 (T), MP9 (CT)	\$1,050 / \$1,250	
	MP7, MP5-SD	\$1,400	
	UMP-45	\$1,200	
	P90	\$2,350	\$300/kill (instead of \$600)
	PP-Bizon	\$1,400	
Rifles	Galil (T) / FAMAS (CT)	\$1,800 / \$1,950	
	AK-47 (T)	\$2,700	One-tap HS with or without helmet
	M4A4 / M4A1-S (CT)	\$2,900	No one-tap HS with helmet
	AUG (CT) / SG 553 (T)	\$3,300 / \$3,000	Scoped
Snipers	SSG 08 (scout)	\$1,700	
	AWP	\$4,750	Guaranteed kill in one shot (upper body)
Grenades	Smoke	\$300	
	Flashbang	\$200	Max 2 per player
	HE Grenade	\$300	
	Molotov (T)	\$400	
	Incendiary (CT)	\$500	
	Decoy	\$50	Mimics gunfire

Table 5: Main equipment prices in CS2 Competitive.

Kill rewards. Killing an opponent pays an amount that depends on the weapon used: \$300 for a pistol or rifle, \$100 for an AWP (overpowered weapon, so nerfed reward), \$600 for an SMG (except P90 at \$300, same logic), \$900 for a shotgun (except XM1014 at \$600), \$1,500 for a knife, \$300 for a grenade. **CT specifics:** a CT killing a T receives an extra \$50, regardless of the weapon or time in the round. This \$50-per-kill bonus gives CT a slight structural economic edge.

Round-win rewards. Each player on the winning team receives:

- **\$3,250** for a win by elimination or by *time-out* (CT);
- **\$3,500** for a win by detonation (T) or defuse (CT). Bomb-based wins thus pay \$250 more than the other modes.

On top of that, \$300 personal goes to the player who plants or defuses the bomb.

Loss bonus and *loss count*. A team that loses a round receives compensation that depends on a team-wide counter called *loss count*:

- the counter starts at 1 at the start of each half;
- it is updated *after* the round: +1 on a loss, *reset to 1* on a win, capped at 4;
- it is also reset to 1 at half-time and at every side switch during overtime.

The bonus schedule is: loss count 0 $\rightarrow$ \$1,400, 1 $\rightarrow$ \$1,900, 2 $\rightarrow$ \$2,400, 3 $\rightarrow$ \$2,900, 4 $\rightarrow$ \$3,400. The first loss of a half thus pays \$1,900 per player.

Plant bonus on a defeat. If the Terrorists *plant* the bomb but still lose the round by defuse, every Terrorist receives an extra \$600 on top of the loss bonus.

Time-out edge case. If the Terrorists lose because the round timer ran out without a plant, the surviving Terrorists receive nothing.

Buy types. Without a formal definition, four economic regimes are usually distinguished:

- **eco:** nothing is bought, or only a pistol;
- **half-buy:** just enough is bought to leave room for a *full-buy* on the next round;
- **force-buy:** everything is spent without reaching a *full-buy*, at the cost of not being able to buy on the next round;
- **full-buy:** everything needed is purchased (rifle, armour, utility).

CT players on a *full-buy* do not always buy the helmet, especially against a T *full-buy*: a helmet does not help against an AK.

B Demo parsing

B.1 Demos and libraries

A demo is a complete recording of a match: player positions, inventory, money, hit points, and the history of every action (shots, damage, purchases, grenades thrown, plants, defuses). One can jump to any point in time and inspect the game state.

HLTV hosts, for every tournament match, a ZIP archive containing the demos of the maps played, automatically downloaded by our scraper. Two Python libraries are used together to read them:

- **demoparser2:** low-level parser giving raw access to the game state at chosen ticks (`parse_ticks`) or to the list of individual events (`parse_event`).
- **awpy:** higher-level Python parser, built on top of `demoparser2`, that directly exposes useful tables (`rounds`, `grenades`, `bomb`, map header, etc.) after a single `parse()` call.

For example, to retrieve the `equipment_value` at *freeze-time* of every round:

```
1 from demoparser2 import DemoParser
2 parser = DemoParser("path/to/demo.dem")
3 wanted_ticks = parser.parse_event("round_freeze_end")["tick"].tolist()
4 df = parser.parse_ticks(
5     ["current_equip_value", "total_rounds_played"],
6     ticks=wanted_ticks,
7 )
```

B.2 Snapshot tick

For every round we capture a *snapshot* of the game state at a specific tick. Taking it at `round_start` would be too early: no purchase has been made yet. Taking it exactly at `freeze_end` is not enough either, because the buy phase (35s) extends beyond the freeze-time (15s): a player can still buy for about 20 extra seconds as long as they stay in the buy zone. A player can also pick up a weapon dropped by a teammate right after movement is unlocked. The chosen compromise shifts the snapshot by two seconds:

$$\text{snapshot_tick} = \text{freeze_end_tick} + 2\text{s} \times 128 \text{ ticks/s}$$

This window leaves enough time for late transactions to settle but remains too short for any duel to have happened: all ten players are still alive, which guarantees a coherent snapshot.

B.3 The "insta-smoke" problem and grenade counting

The 2-second offset of the snapshot (`freeze_end + 2s`) creates a problem for grenade counting. During the freeze-time, players can only buy: they cannot throw anything. But as soon as the freeze ends, they can immediately execute "insta-smokes": timed smokes thrown in the first second of the round to cover a spawn exit or block an angle. By `snapshot_tick`, those grenades are no longer in the player's

inventory. A naive read of the inventory would systematically under-estimate the utility of teams that rely on fast executions.

The fix combines two sources of information: the inventory at `snapshot_tick` and the `awpy.grenades` table filtered over the interval `[freeze_end, snapshot_tick]`, which lists every grenade thrown in that two-second window. A second issue then appears: `demoparser2` removes thrown grenades from the inventory with a slight delay (~ 0.1 s), so the same grenade can momentarily appear both in the inventory and in the throw table. We therefore cap the per-player total by the maximum quantities buyable in one round: one smoke, one molo, one HE, two flashes.

```

1 def count_items(snapshot_data, grenade_df, item):
2     items = {item} if isinstance(item, str) else set(item)
3     max_per_player = 2 if "Flashbang" in items else 1
4     thrown_by_player = (grenade_df[grenade_df["grenade_type"].isin(items)]
5                          .groupby("thrower").size())
6     total = 0
7     for _, player in snapshot_data.iterrows():
8         inv_count = sum(1 for x in player["inventory"] if x in items)
9         player_thrown = thrown_by_player.get(player["name"], 0)
10        total += min(inv_count + player_thrown, max_per_player)
11    return int(total)

```

B.4 Quality fixes applied

The `clean_rounds.py` script concatenates all per-map CSVs produced by the parser, applies the fixes described below, and produces `df_rounds_raw.csv`.

Corrupted demos. Four demos are structurally corrupted at the source on HLTV (the parser either fails silently or produces inconsistent scores). They are removed before any processing: *mouz vs liquid inferno*, *mouz vs furia mirage*, *parivision vs astralis dust2*, *mouz vs parivision dust2*.

Phantom rounds. Some demos start with an artificial "round 0" caused by a *warmup*, a knife round, or a disconnection/reforfeit. Those phantom rounds shift every subsequent `round_number`. For each match we locate the real R1 as the first row whose "pistol shape" is satisfied — both teams' `money_total` in `[$4,600, $6,500]`, a deliberately wide range to accommodate the *fissure-playground-2* case described below. Preceding rows are dropped, and the same logic is applied to find R13 at expected position 12. The `round_number` are then renumbered from 1 to n .

Snapshot drift in *fissure-playground-2*. An entire tournament, *fissure-playground-2*, produces demos in which the recording does not start exactly at round 1 but about ten seconds later: the `freeze_end` is missed and the `start_tick` falls in the middle of the round. The `money_total` of both sides is then no longer exactly \$5,000 but drifts in a small range (e.g. 4800-4800, 5400-5300). This is precisely why the "pistol shape" is defined loosely as `[$4,600, $6,500]` rather than strictly \$5,000. The 92 affected rounds are kept.

4v5 pistols. Another pathological case occurs when a player is missing on one side at pistol time (technical absence, *kick*): $4 \times \$800 = \$3,200$ cash plus $4 \times \$200 = \800 pistols, i.e. exactly \$4,000 on one side. This is not a real playable round (5 vs 4 skews the outcome drastically). It is detected by `money_total == $4,000` at R1 or R13 and dropped: 16 rounds concerned (9 R1, 7 R13).

C HLTV scraping

For each target tournament, the scraper visits three types of pages:

- the *event page* to extract the tournament name, dates, map pool and list of matches;
- the *match result page* for the official scores and the demo download link;
- the *team statistics page* of each team.

Team statistics are scraped along several dimensions: by **map** (Mirage, Inferno, Dust2..., or *global* all-maps), by **side** (CT, T, both), by **time window** (30 d, 90 d, 6 months) and by **opponent tier** (Top 5, Top 10, Top 20, Top 30, Top 50). From those we extract the main indicators: rating, ADR, round-win %, flash-assists, pistol-win %, etc.

"Top" propagation rule. HLTV does not always publish every combination: a team outside the Top 30 may not have any "vs Top 5" statistics. To limit the holes we apply a downward propagation rule: Top 5 data fills Top 10, which fills Top 20, and so on down to Top 50. The propagation never goes the other way: if Top 5 is empty, we do not fill it with Top 20 data, which would paint a falsely optimistic picture.

HLTV NaN handling. Not every (team, map, side, window, top) combination is available on HLTV. We fill the resulting NaN with three successive fallbacks (Table 6). These choices are arbitrary: we could just as well have replaced every NaN with a single neutral value (e.g. 1.00 for the rating).

Step	Mechanism	NaN filled	Remaining
Initial	Empty cells after the join	—	204,000 (13.5 %)
1. map → global	All-maps stat when the map-specific is missing	−106,000	98,000
2. cross-window	30 d → 90 d → 6 months if empty	−33,000	65,000
3. expanding median	Median per column over past values	−65,000	0

Table 6: Progressive HLTV NaN filling.